

```

01. #define PIN_MQ9 A0 // Номер пина подключения data-выхода датчика угарного и горючих
    газов
02. #define PIN_MQ9_HEATER 13 // Номер пина подключения нагревателя (heater) датчика
    угарного и горючих газов
03. #define ONE_WIRE_BUS 11 // Номер пина подключения data-выхода датчика температуры
04. #define TEMPERATURE_PRECISION 12 // Разрешение датчика (может быть от 9 до 12 бит,
    определение точности и скорости измерения)
05. #define _LCD_TYPE 1 // Тип подключения дисплея (1 - по шине I2C, 2 -
    десятиконтактное)
06. #define PIN_IRRECV 9 // Номер пина подключения data-выхода ИК-приёмника HX1838
07. #define DISPLAY_ADDRESS 0x27 // Адрес LCD-дисплея
08. #define DISPLAY_LENGTH 16 // Длина LCD-дисплея
09. #define DISPLAY_WIDTH 2 // Ширина LCD-дисплея
10.
11. #include <ТройкаMQ.h> // Библиотека для работы с датчиком угарного и горючих газов
    MQ-9
12. #include <LCD_1602_RUS_ALL.h> // Библиотека для работы с LCD-экраном 1602A
13. #include <DallasTemperature.h> // Библиотека для работы с датчиком температуры
    DS18B20
14. #include <OneWire.h> // Библиотека для работы с устройствами, работающими на шине и
    по протоколу 1-Wire
15. #include <IRremote.h> // Библиотека для работы с ИК-приемниками
16.
17. MQ9 mq9(PIN_MQ9, PIN_MQ9_HEATER); // Объявление параметров для датчика газа (номер
    пина data-выхода, номер пина нагревателя)
18. LCD_1602_RUS <LiquidCrystal_I2C> lcd(DISPLAY_ADDRESS, DISPLAY_LENGTH, DISPLAY_WIDTH);
    // Объявление параметров нашего LCD-экрана (адрес дисплея, длина, ширина)
19. OneWire oneWire(ONE_WIRE_BUS); // Объявление параметров устройства, работающем по
    протоколу 1-Wire (номер пина data-выхода)
20. DallasTemperature sensors(&oneWire); // Связывание функции библиотеки
    DallasTemperature с нашим 1-Wire устройством (датчик температуры DS18B20)
21. DeviceAddress tempDeviceAddress; // Объявление переменной для хранения адреса
    найденного датчика
22. IRrecv irrecv(PIN_IRRECV); // Создание объекта IRrecv для работы с ИК-приёмником с
    использованием пина PIN_IRRECV
23. decode_results results; // Переменная для хранения и расшифровки полученных данных от
    ИК-приемника
24.
25. int numberOfDevices; // Объявление переменной для получения количества подключённых
    датчиков
26. int CalibrationEnd = 0; // Объявление переменной для флага завершения калибровки
    датчика MQ-9
27. int SignalingFlag = 0; // Объявление переменной для подсчёта количества нажатий на
    пульте кнопки "OK"
28.
29. byte customChar[] = { // Создание нового символа (значок градусуса)
30.     B11110,
31.     B10010,
32.     B10010,
33.     B11110,
34.     B00000,
35.     B00000,
36.     B00000,
37.     B00000
38. };
39.
40. byte customChar1[] = { // Создание нового символа (значок Цельсия)
41.     B11111,
42.     B11111,
43.     B11000,
44.     B11000,
45.     B11000,

```

```

46.     B11000,
47.     B11111,
48.     B11111
49. };
50.
51. byte customChar2[] = { // Создание нового символа (значок Фаренгейта)
52.     B11111,
53.     B11111,
54.     B11000,
55.     B11000,
56.     B11111,
57.     B11111,
58.     B11000,
59.     B11000
60. };
61.
62. byte customChar3[] = { // Создание нового символа (значок разделительный)
63.     B01110,
64.     B01110,
65.     B01110,
66.     B01110,
67.     B01110,
68.     B01110,
69.     B01110,
70.     B01110
71. };
72.
73. byte customChar4[] = { // Создание нового символа (значок шкалы прогресса)
74.     B11111,
75.     B11111,
76.     B11111,
77.     B11111,
78.     B11111,
79.     B11111,
80.     B11111,
81.     B11111
82. };
83.
84. void setup() { // Функция, которая вызывается при старте скетча
85.     irrecv.enableIRIn(); // Инициализация ИК-приёмника HX1838
86.     pinMode(8, OUTPUT); // Устанавливаем номер пина для подачи на него напряжения (для
звонка)
87.     mq9.cycleHeat(); // Запуск термоцикла
88.     lcd.init(); // Инициализация LCD-экрана
89.     lcd.backlight(); // Включение подсветки на LCD-экране
90.     sensors.begin(); // Инициализация датчика температуры DS18B20
91.     numberOfDevices = sensors.getDeviceCount(); // Вызов функции подсчёта количества
датчиков и запись их количества в переменную
92.     sensors.setResolution(tempDeviceAddress, TEMPERATURE_PRECISION); // Вызов функции
для определения разрешения датчика (точность и скорость измерения)
93. }
94.
95. void printOption() { // Функция для вывода сообщения о выборе опции на ИК-пульте
96.     lcd.clear(); // Очистка экрана
97.     lcd.setCursor(5,0); // Перемещение курсора на экране в позицию (5,0)
98.     lcd.print("Select"); // Печать на экране "Select"
99.     lcd.setCursor(5,1); // Перемещение курсора на экране в позицию (5,1)
100.    lcd.print("option:"); // Печать на экране "option:"
101. }
102.
103. void printWarning() { // Функция для вывода сообщения об опасности
104.     lcd.clear(); // Очистка экрана

```

```

105.     lcd.setCursor(4,0); // Перемещение курсора на экране в позицию (4,0)
106.     lcd.print("WARNING!"); // Печать на экране "WARNING!"
107.     lcd.setCursor(0,1); // Перемещение курсора на экране в позицию (0,1)
108.     lcd.print("FOLLOW THE EXIT!"); // Печать на экране "FOLLOW THE EXIT!"
109.     for(int i = 0; i < 30; i++) { // Цикл для звонка (30 звуковых сигналов)
110.         digitalWrite(8, HIGH); // На пин со звонком подаётся напряжение
111.         delay(200); // Ожидание
112.         digitalWrite(8, LOW); // Напряжение с пина убирается
113.         delay(200); // Ожидание
114.     }
115.     delay(1000); // Ожидание
116. }
117.
118. void printCalibrationProgress() { // Функция для вывода на экран процесса калибровки
    датчика MQ-9
119.
120.     lcd.clear(); // Очистка экрана
121.     lcd.createChar(4, customChar4); // Создание символа и присваивание ему номера
    (значок шкалы прогресса)
122.     lcd.setCursor(1,0); // Перемещение курсора на экране в позицию (1,0)
123.     lcd.print("CALIBRATION..."); // Печать на экране "CALIBRATION..."
124.     for (int i = 0; i < 16; i++) { // Цикл для печати шкалы прогресса калибровки
    датчика MQ-9
125.         lcd.setCursor(i,1); // Перемещение курсора на экране в позицию (i,1)
126.         lcd.write(4); // Печать на экране значка шкалы прогресса
127.         delay(1000); // Ожидание
128.     }
129. }
130.
131. void printCreatorName() { // Функция для вывода на экран разработчика устройства
132.     lcd.clear(); // Очистка экрана
133.     lcd.setCursor(4,0); // Перемещение курсора на экране в позицию (4,0)
134.     lcd.print("Made by"); // Печать на экране "Made by"
135.     lcd.setCursor(2,1); // Перемещение курсора на экране в позицию (2,1)
136.     lcd.print("PIOTR SHARAI"); // Печать на экране "PIOTR SHARAI"
137. }
138.
139. int Calibration(){ // Функция для калибровки датчика MQ-9
140.     int CalibrationFlag1 = 0; // Объявление флага для одноразового вывода сообщения о
    начале калибровки датчика
141.     int CalibrationFlag2 = 0; // Объявление флага для одноразового вывода сообщения о
    завершении калибровки датчика
142.     printCreatorName(); // Вызов функции для вывода на экран разработчика устройства
143.     while (CalibrationFlag2 != 1) // Цикл для отслеживания полной калибровки датчика
    MQ-9
144.     {
145.         if (!mq9.isCalibrated() && mq9.atHeatCycleEnd() && CalibrationFlag1 == 0) // Если
    прошёл интервал нагрева датчика, калибровка не была завершена и CalibrationFlag1 == 0
146.         {
147.             for(int i = 0; i < 1; i++) // Цикл для звонка (1 звуковой сигнал)
148.             {
149.                 digitalWrite(8, HIGH); // На пин со звонком подаётся напряжение
150.                 delay(300); // Ожидание
151.                 digitalWrite(8, LOW); // Напряжение с пина убирается
152.                 delay(300); // Ожидание
153.             }
154.             lcd.clear(); // Очистка экрана
155.             lcd.setCursor(2,0); // Перемещение курсора на экране в позицию (2,0)
156.             lcd.print("Gas sensor"); // Печать на экране "Gas sensor"
157.             lcd.setCursor(2,1); // Перемещение курсора на экране в позицию (2,1)
158.             lcd.print("calibration!"); // Печать на экране "calibration!"
159.             delay(3000); // Ожидание

```

```

160.
161.     mq9.calibrate(); // Вызов функции для калибровки датчика MQ-9 на чистом воздухе
162.     printCalibrationProgress(); // Вызов функции для вывода на экран процесса
калибровки датчика MQ-9
163.     CalibrationFlag1 = 1; // Флаг CalibrationFlag1 устанавливается в 1
164. }
165. if (mq9.isCalibrated() && CalibrationFlag2 == 0) { // Если калибровка была
завершена и CalibrationFlag2 == 0
166.     for(int i = 0; i < 2; i++) // Цикл для звонка (2 звуковых сигнала)
167.     {
168.         digitalWrite(8, HIGH); // На пин со звонком подаётся напряжение
169.         delay(300); // Ожидание
170.         digitalWrite(8, LOW); // Напряжение с пина убирается
171.         delay(300); // Ожидание
172.     }
173.     lcd.clear(); // Очистка экрана
174.     lcd.setCursor(2,0); // Перемещение курсора на экране в позицию (2,0)
175.     lcd.print("Calibration"); // Печать на экране "Calibration"
176.     lcd.setCursor(2,1); // Перемещение курсора на экране в позицию (2,1)
177.     lcd.print("completed!"); // Печать на экране "completed!"
178.     CalibrationFlag2 = 1; // Переменная CalibrationFlag2 становится равной 1
179.     delay(3000); // Ожидание
180.     printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
пульте
181.     mq9.cycleHeat(); // Запуск термоцикла
182. }
183. }
184. return 1; // Функция возвращает 1
185. }
186. void printLPG(){ // Функция для вывода на экран уровня LPG с датчика MQ-9
187.     float LPG = 0; // Переменная для записи уровня LPG с датчика MQ-9
188.     String LPGstr = ""; // Переменная для вывода на экран уровня LPG с датчика MQ-9
189.
190.     LPG = mq9.readLPG(); // Запись в переменную уровня сжиженных углеводородных газов
(LPG)
191.     LPGstr = String(LPG, 0); // Запись в переменную для вывода на экран уровня
сжиженных углеводородных газов (LPG)
192.     lcd.clear(); // Очистка экрана
193.     lcd.setCursor(6,0); // Перемещение курсора на экране в позицию (6,0)
194.     lcd.print("LPG:"); // Печать на экране "LPG:"
195.     lcd.setCursor(3,1); // Перемещение курсора на экране в позицию (3,1)
196.     lcd.print(LPGstr); // Печать на экране переменной LPGstr
197.     lcd.setCursor(8,1); // Перемещение курсора на экране в позицию (8,1)
198.     lcd.print("ppm"); // Печать на экране "ppm"
199. }
200. void printCO(){ // Функция для вывода на экран уровня CO с датчика MQ-9
201.     float CO = 0; // Переменная для записи уровня CO с датчика MQ-9
202.     String COstr = ""; // Переменная для вывода на экран уровня CO с датчика MQ-9
203.     CO = mq9.readCarbonMonoxide(); // Запись в переменную уровня угарного газа (CO)
204.     COstr = String(CO, 0); // Запись в переменную для вывода на экран уровня угарного
газа (CO)
205.     lcd.clear(); // Очистка экрана
206.     lcd.setCursor(6,0); // Перемещение курсора на экране в позицию (6,0)
207.     lcd.print("CO:"); // Печать на экране "CO:"
208.     lcd.setCursor(3,1); // Перемещение курсора на экране в позицию (3,1)
209.     lcd.print(COstr); // Печать на экране переменной COstr
210.     lcd.setCursor(8,1); // Перемещение курсора на экране в позицию (8,1)
211.     lcd.print("ppm"); // Печать на экране "ppm"
212. }
213. void printMethane(){ // Функция для вывода на экран уровня Methane с датчика MQ-9
214.     float Methane = 0; // Переменная для записи уровня Methane с датчика MQ-9

```

```

215.   String Methanestr = ""; // Переменная для вывода на экран уровня Methane с датчика
MQ-9
216.   Methane = mq9.readMethane(); // Запись в переменную уровня метана (Methane)
217.   Methanestr = String(Methane, 0); // Запись в переменную для вывода на экран уровня
метана (Methane)
218.   lcd.clear(); // Очистка экрана
219.   lcd.setCursor(4,0); // Перемещение курсора на экране в позицию (4,0)
220.   lcd.print("Methane:"); // Печать на экране "Methane:"
221.   lcd.setCursor(3,1); // Перемещение курсора на экране в позицию (3,1)
222.   lcd.print(Methanestr); // Печать на экране переменной Methanestr
223.   lcd.setCursor(8,1); // Перемещение курсора на экране в позицию (8,1)
224.   lcd.print("ppm"); // Печать на экране "ppm"
225. }
226.
227. void printTemperatureC(DeviceAddress deviceAddress){ // Функция для вывода на экран
температуры в цельсиях
228.   float tempC = 0; // Переменная для записи уровня температуры в цельсиях с датчика
температуры
229.   String tempCstr = ""; // Переменная для вывода на экран уровня температуры в
цельсиях с датчика температуры
230.   lcd.clear(); // Очистка экрана
231.   lcd.createChar(0, customChar); // Создание символа и присваивание ему номера
(значок грудуса)
232.   lcd.createChar(1, customChar1); // Создание символа и присваивание ему номера
(значок Цельсия)
233.
234.   lcd.setCursor(5,0); // Перемещение курсора на экране в позицию (5,0)
235.   lcd.print("Temp"); // Печать на экране "Temp"
236.   lcd.write(0); // Печать на экране значка градуса
237.   lcd.write(1); // Печать на экране значка Цельсия
238.
239.   if(sensors.getTempC(deviceAddress) > 0){ // Если температура, полученная с датчика,
больше 0
240.     lcd.setCursor(5,1); // Перемещение курсора на экране в позицию (5,1)
241.     lcd.print("+"); // Печать на экране значка "+"
242.   }
243.   if(sensors.getTempC(deviceAddress) < 0){ // Если температура, полученная с датчика,
меньше 0
244.     lcd.setCursor(5,1); // Перемещение курсора на экране в позицию (5,1)
245.     lcd.print(" "); // Печать на экране пустого значка
246.   }
247.   lcd.setCursor(6,1); // Перемещение курсора на экране в позицию (6,1)
248.   tempC = sensors.getTempC(deviceAddress); // Запись в переменную температуры в
цельсиях
249.   tempCstr = String(tempC, 2); // Запись в переменную для вывода на экран температуры
в цельсиях
250.   lcd.print(tempCstr); // Печать на экране значения температуры в цельсиях,
полученной с датчика температуры
251. }
252. void printTemperatureF(DeviceAddress deviceAddress){ // Функция для вывода на экран
температуры в фаренгейтах
253.   float tempF = 0; // Переменная для записи уровня температуры в фаренгейтах с
датчика температуры
254.   String tempFstr = ""; // Переменная для вывода на экран уровня температуры в
фаренгейтах с датчика температуры
255.   lcd.clear(); // Очистка экрана
256.   lcd.createChar(0, customChar); // Создание символа и присваивание ему номера
(значок грудуса)
257.   lcd.createChar(2, customChar2); // Создание символа и присваивание ему номера
(значок Фаренгейта)
258.
259.   lcd.setCursor(5,0); // Перемещение курсора на экране в позицию (5,0)

```

```

260.    lcd.print("Temp"); // Печать на экране "Temp"
261.    lcd.write(0); // Печать на экране значка градуса
262.    lcd.write(2); // Печать на экране значка Фаренгейта
263.
264.    if(sensors.getTempC(deviceAddress) > 0){ // Если температура, полученная с датчика,
        больше 0
265.        lcd.setCursor(5,1); // Перемещение курсора на экране в позицию (5,1)
266.        lcd.print("+"); // Печать на экране значка "+"
267.    }
268.    if(sensors.getTempC(deviceAddress) < 0){ // Если температура, полученная с датчика,
        меньше 0
269.        lcd.setCursor(5,1); // Перемещение курсора на экране в позицию (5,1)
270.        lcd.print(" "); // Печать на экране пустого значка
271.    }
272.    lcd.setCursor(6,1); // Перемещение курсора на экране в позицию (6,1)
273.    tempF = sensors.getTempF(deviceAddress); // Запись в переменную температуры в
        фаренгейтах
274.    tempFstr = String(tempF, 2); // Запись в переменную для вывода на экран температуры
        в фаренгейтах
275.    lcd.print(tempFstr); // Печать на экране значения температуры в фаренгейтах,
        полученной с датчика температуры
276. }
277. void loop() // Функция, команды в которой будут повторяться, пока включена плата
        Arduino
278. {
279.     if (CalibrationEnd != 1) // Если калибровка датчика MQ-9 не была выполнена
280.     {
281.         CalibrationEnd = Calibration();
282.     }
283.     if (CalibrationEnd == 1) // Если калибровка датчика MQ-9 была выполнена
284.     {
285.         sensors.requestTemperatures(); // Запрос датчику для измерения температуры
286.
287.         for(int i=0;i<numberOfDevices; i++) // Цикл для получения показаний с датчика
288.         {
289.             if(sensors.getAddress(tempDeviceAddress, i)) // Если была установлена
                корректная связь с датчиком
290.             {
291.                 if((sensors.getTempC(tempDeviceAddress)) >= 30 ||
                ((sensors.getTempF(tempDeviceAddress)) >= 86)){ // Если показания температуры
                превышают норму
292.                     printWarning(); // Вызов функции для сообщения об опасности
293.                     delay(500); // Ожидание
294.                     printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
                        пульте
295.                 }
296.             }
297.         }
298.         float LPG = mq9.readLPG(); // Запись в переменную уровня сжиженных углеводородных
                газов (LPG)
299.         float Methane = mq9.readMethane(); // Запись в переменную уровня метана (CH4)
300.         float CO = mq9.readCarbonMonoxide(); // Запись в переменную уровня угарного газа
                (CO)
301.         if (LPG >= 60 && Methane >= 60 && CO >= 60) // Если уровень газов превысит норму
302.         {
303.             printWarning(); // Вызов функции для сообщения об опасности
304.             delay(500); // Ожидание
305.             printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
                пульте
306.         }
307.         if (irrecv.decode(&results)) { // Если с ИК-пульта пришёл сигнал
308.             if (results.value == 0xFFA25D){ // Если с ИК-пульта пришёл сигнал кнопки "1"

```

```

309.         for (int i = 0; i < 7; i++){ // Цикл на 7 повторений
310.             sensors.requestTemperatures(); // Запрос датчику для измерения температуры
311.             for(int i=0;i<numberOfDevices; i++) // Цикл для получения показаний с
датчика температуры
312.             {
313.                 if(sensors.getAddress(tempDeviceAddress, i)) // Если была установлена
корректная связь с датчиком температуры
314.                 {
315.                     if((sensors.getTempC(tempDeviceAddress)) >= 30){ // Если показания
температуры превышают норму
316.                         printWarning(); // Вызов функции для сообщения об опасности
317.                         delay(500); // Ожидание
318.                         printOption(); // Вызов функции для вывода сообщения о выборе опции
на ИК-пульте
319.                         break; // Выход из цикла
320.                     }
321.                     printTemperatureC(tempDeviceAddress); // Вызов функции получения
температуры в цельсиях с датчика температуры и её вывод на LCD-экран
322.                 }
323.             }
324.             delay (1000); // Ожидание
325.         }
326.         delay (3000); // Ожидание
327.         printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
пульте
328.     }
329.     if (results.value == 0xFF629D){ // Если с ИК-пульта пришёл сигнал кнопки "2"
330.         for (int i = 0; i < 7; i++){ // Цикл на 7 повторений
331.             sensors.requestTemperatures(); // Запрос датчику для измерения температуры
332.             for(int i=0;i<numberOfDevices; i++) // Цикл для получения показаний с
датчика температуры
333.             {
334.                 if(sensors.getAddress(tempDeviceAddress, i)) // Если была установлена
корректная связь с датчиком температуры
335.                 {
336.                     if((sensors.getTempF(tempDeviceAddress)) >= 86){ // Если показания
температуры превышают норму
337.                         printWarning(); // Вызов функции для сообщения об опасности
338.                         delay(500); // Ожидание
339.                         printOption(); // Вызов функции для вывода сообщения о выборе опции
на ИК-пульте
340.                         break; // Выход из цикла
341.                     }
342.                     printTemperatureF(tempDeviceAddress); // Вызов функции получения
температуры в фаренгейтах с датчика температуры и её вывод на LCD-экран
343.                 }
344.             }
345.             delay (1000); // Ожидание
346.         }
347.         delay (3000); // Ожидание
348.         printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
пульте
349.     }
350.     if (results.value == 0xFFE21D){ // Если с ИК-пульта пришёл сигнал кнопки "3"
351.         for (int i = 0; i < 7; i++){ // Цикл на 7 повторений
352.             if(mq9.readLPG() >= 60){ // Если уровень LPG превышет норму
353.                 printWarning(); // Вызов функции для сообщения об опасности
354.                 delay(500); // Ожидание
355.                 printOption(); // Вызов функции для вывода сообщения о выборе опции на
ИК-пульте
356.                 break; // Выход из цикла
357.             }

```

```

358.         printLPG(); // Вызов функции для вывода на экран уровня LPG с датчика MQ-9
359.         delay (1000); // Ожидание
360.     }
361.     delay (3000); // Ожидание
362.     printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
пульт
363.     }
364.     if (results.value == 0xFF22DD){ // Если с ИК-пульта пришёл сигнал кнопки "4"
365.         for (int i = 0; i < 7; i++){ // Цикл на 7 повторений
366.             if(mq9.readCarbonMonoxide() >= 60){ // Если уровень CO превышет норму
367.                 printWarning(); // Вызов функции для сообщения об опасности
368.                 delay(500); // Ожидание
369.                 printOption(); // Вызов функции для вывода сообщения о выборе опции на
ИК-пульт
370.                 break; // Выход из цикла
371.             }
372.             printCO(); // Вызов функции для вывода на экран уровня CO с датчика MQ-9
373.             delay (1000); // Ожидание
374.         }
375.         delay (3000); // Ожидание
376.         printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
пульт
377.     }
378.     if (results.value == 0xFF02FD){ // Если с ИК-пульта пришёл сигнал кнопки "5"
379.         for (int i = 0; i < 7; i++){ // Цикл на 7 повторений
380.             if(mq9.readMethane() >= 60){ // Если уровень Methane превышет норму
381.                 printWarning(); // Вызов функции для сообщения об опасности
382.                 delay(500); // Ожидание
383.                 printOption(); // Вызов функции для вывода сообщения о выборе опции на
ИК-пульт
384.                 break; // Выход из цикла
385.             }
386.             printMethane(); // Вызов функции для вывода на экран уровня Methane с
датчика MQ-9
387.             delay (1000); // Ожидание
388.         }
389.         delay (3000); // Ожидание
390.         printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
пульт
391.     }
392.     if (results.value == 0xFF38C7){ // Если с ИК-пульта пришёл сигнал кнопки "OK"
393.         SignalingFlag++; // Увеличение на 1 переменной для подсчёта количества
нажатий на пульте кнопки "OK"
394.         delay(300); // Ожидание
395.         if (SignalingFlag == 3){ // Если SignalingFlag == 3, т.е кнопка "OK" была
нажата 3 раза
396.             printWarning(); // Вызов функции для сообщения об опасности
397.             delay(500); // Ожидание
398.             printOption(); // Вызов функции для вывода сообщения о выборе опции на ИК-
пульт
399.             SignalingFlag = 0; // Обнуление переменной SignalingFlag
400.         }
401.     }
402.     if (mq9.isCalibrated() && mq9.atHeatCycleEnd()) // Если прошёл интервал нагрева
датчика, и калибровка была совершена
403.     {
404.         mq9.cycleHeat(); // Запуск термоцикла
405.     }
406.     irrecv.resume(); // Возобновление приёма сигналов с ИК-пульта
407. }
408. }
409. }

```